UCS503 - Software Engineering

PatchWatch

Software Dependency Security and Risk Dashboard

Submitted by:

1024030004 Diya Gupta
1024030007 Durva Garg
1024030010 Mysha Bansal

Group: **3C11** B.E. **3rd Year**, COE

Submitted to:

Ms. Paramveer Kaur

Prototype Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **1**

1.1 Background and Context 1

1.2 Motivation 1

1.3 Problem Statement 2

1.4 Project Objectives 2

2 Methodology and Design **3**

2.1 Overall Approach 3

2.2 System Architecture 4

2.2.1 High-Level Design 4

2.2.2 Request Flow when a User Runs a Scan 4

2.3 Technical Stack 4

2.4 UML Use Case Diagram 6

2.5 Data Model 6

2.6 Dependency Check States 6

2.7 Risk Score Design 7

3 Implementation **8**

3.1 Backend 8

3.1.1 API Endpoints 8

3.1.2 Parser 8

3.1.3 Vulnerability Checker 9

3.1.4 Persistence 9

3.2 Frontend 10

3.3 Running the Project 10

4 Results and Testing **14**

4.1 Testing Strategy 14

4.2 Automated Test Results 14
4.3 Manual End-to-End Test Plan 15
4.4 Discussion 15

5 Conclusion and Future Work **17**
5.1 Conclusion 17
5.2 Future Work 17
5.3 Lessons Learned 17

References **19**

Chapter 1

Introduction

1.1 Background and Context

Almost no modern web application is written from scratch. A typical JavaScript project pulls in dozens of direct dependencies from npm, and each of those pulls in more. Every one of these packages is code the developer did not write and usually has not read. When a security flaw is found in one of them, it is published in a public vulnerability database, but nothing forces the developer to find out. In practice the project keeps running with a known-vulnerable version until someone happens to notice.

The data needed to catch this already exists. OSV.dev is an open vulnerability database that can be queried by package name and version. What is missing for a student or small team is a simple tool that takes the dependency list they already have, checks it, and presents the result in a form that shows what is risky and what to fix first.

PatchWatch is a web application built for that gap. A user uploads a `package.json`, the backend extracts the dependencies, each one is checked against OSV.dev, the findings are stored in a database, and a dashboard shows the severity breakdown, a risk score, the vulnerable packages, and the version that fixes each problem.

1.2 Motivation

In the Software Engineering course we had to choose a real problem and build it the way a software team would: a proposal, design diagrams, incremental implementation, and testing. Dependency vulnerabilities are a problem every one of us runs into as soon as we use `npm install`, so the topic was easy to relate to. We also wanted a project where the core feature talks to a real external service instead of a hard-coded demo. The work was approached collaboratively across interface design, backend development, integration, documentation, and testing. The frontend and backend were developed in parallel where possible, then integrated so that scan results from the API could be displayed by the dashboard.

1.3 Problem Statement

Developers do not have a quick, visual way to learn which of their project's dependencies have known vulnerabilities. Checking by hand means looking up every package in a database, which does not scale. Raw scanner output is also hard to act on because it does not say which issue to fix first. There is no history either, so a team cannot tell whether the project is getting safer or riskier over time.

1.4 Project Objectives

The status of each objective is stated as it is in the code today, not as it was planned.

1. Let a user upload a `package.json` and extract its runtime and development dependencies.
2. Check every dependency against the live OSV.dev API and normalize the severity of each finding.
3. Persist each scan and its findings in a database so that results survive a server restart and scan history is available.
4. Present the results on a dashboard with summary cards, a severity-coded dependency table, priority fixes, a risk score, and a scan-history chart.
5. Provide user accounts and multiple projects.
6. License analysis, outdated-version detection, and `package-lock.json` / `requirements.txt` support.

Chapter 2

Methodology and Design

2.1 Overall Approach

We followed an iterative and incremental model. Instead of building all of the frontend, then all of the backend, we built one working vertical slice at a time and made each slice run end to end before starting the next. The intended order was: a frontend that runs, a minimal backend API, one frontend feature connected to the backend, database persistence, the scanner, and then the connection of scanning, database, API, and dashboard.

One consequence of this approach was that the frontend team was able to work ahead of the backend. The login, registration, and project management screens were built against a mocked layer so the interface could be completed while the backend only supported scanning. The mock is isolated in two files and documented in the code, so it can be replaced by real endpoints later without changing the pages that use it.

Phase	**What was built**
1	Repository skeleton, frontend UI prototype (dashboard layout, stat cards, sample vulnerability table with hard-coded data)
2	FastAPI application with root and health endpoints; `package.json` parser
3	OSV.dev integration (`vuln_checker`); first real scan end to end
4	SQLite persistence with SQLAlchemy models; scan history endpoints
5	Frontend rebuilt in React with routing; dashboard driven by real scan data; mocked auth and projects
6	Automated tests, one-click start scripts, documentation

Table 2.1: Build phases in implementation order. The table records the sequence of work rather than calendar dates.

2.2 System Architecture

2.2.1 High-Level Design

PatchWatch has three parts: a React frontend that runs in the browser, a FastAPI backend that owns all data and business logic, and an external vulnerability source, OSV.dev. The browser never contacts OSV.dev directly; it only talks to our backend. This keeps the external call, the error handling, and the severity normalization in one place.

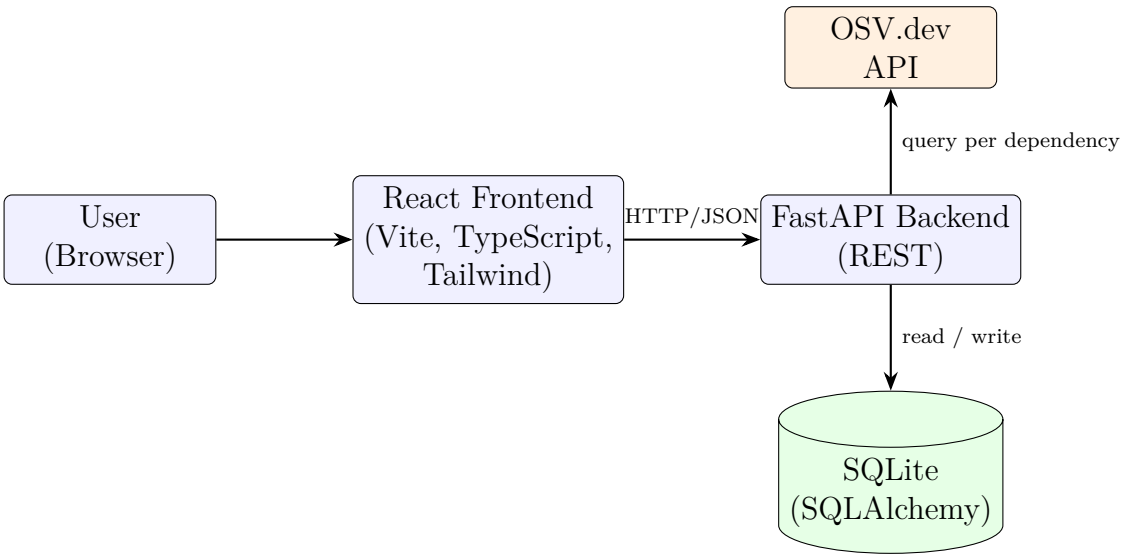


Figure 2.1: High-level architecture of PatchWatch.

2.2.2 Request Flow when a User Runs a Scan

Unlike a background-job design, a scan in PatchWatch is processed inside a single request: the upload does not return until every dependency has been checked and the result is stored. This is acceptable for the size of a typical `package.json` because the OSV lookups are issued concurrently, and the frontend allows up to 60 seconds for the response.

2.3 Technical Stack

- **Backend:** Python 3.10+, FastAPI, Uvicorn, SQLAlchemy 2.0, httpx (async HTTP client), python-multipart.
- **Database:** SQLite, created automatically on first run.
- **Frontend:** React 19, TypeScript, Vite, Tailwind CSS 4, React Router, Axios, lucide-react icons.
- **External data source:** OSV.dev vulnerability API (npm ecosystem).
- **Testing:** pytest with FastAPI's `TestClient`.

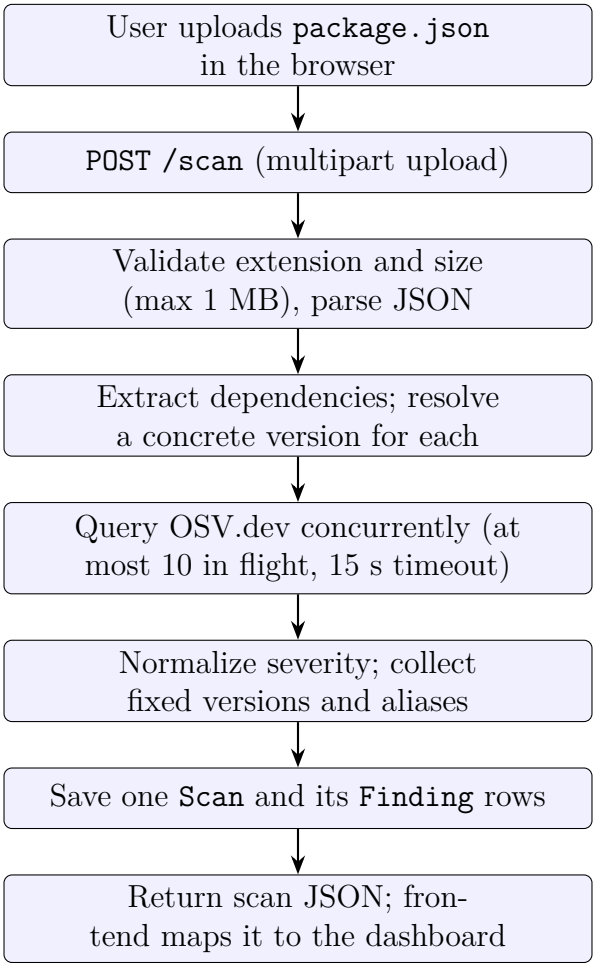


Figure 2.2: Request flow for a scan.

- **Tooling:** Git and GitHub for version control; `start.bat` / `start.sh` scripts that install dependencies and launch both servers.

2.4 UML Use Case Diagram

Because accounts exist only in the browser at this stage, there is a single actor, the user (a developer). OSV.dev is shown as an external system the backend depends on.

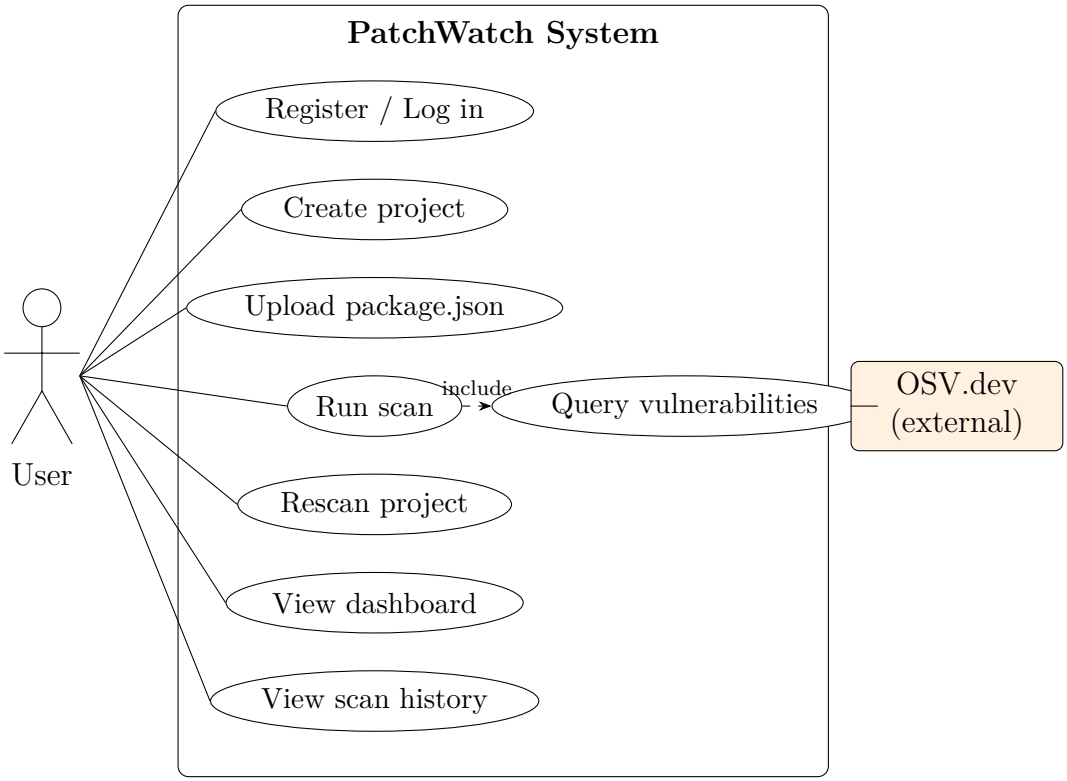


Figure 2.3: Use case diagram of PatchWatch.

2.5 Data Model

The database has two tables. A `Scan` stores the summary of one upload, and each `Finding` stores one vulnerability found for one package version in that scan. Deleting a scan deletes its findings.

2.6 Dependency Check States

Each dependency in the uploaded file ends up in exactly one of four states. This is what lets the system report *why* a dependency was not checked instead of silently ignoring it.

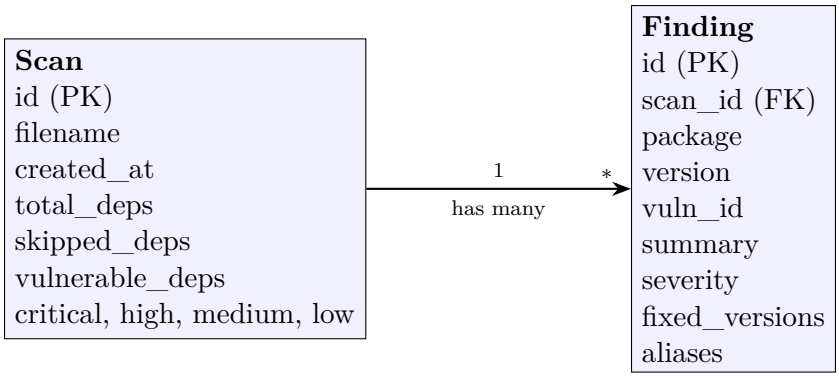


Figure 2.4: Entity–relationship diagram.

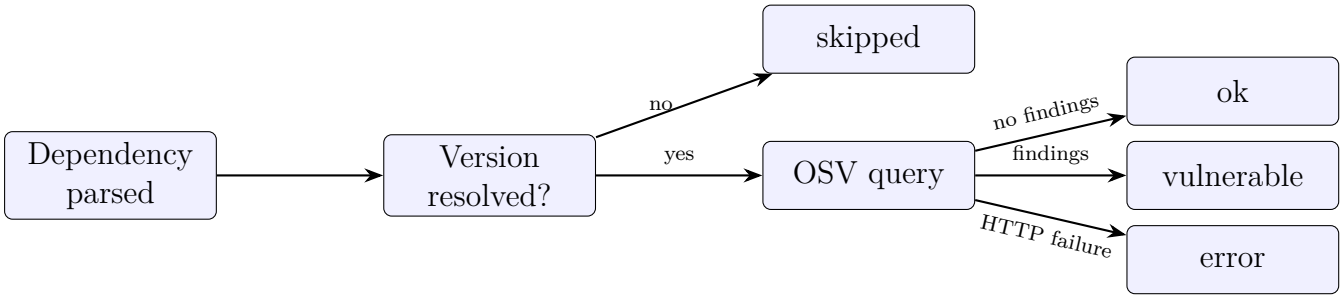


Figure 2.5: States a dependency can end in after a scan.

2.7 Risk Score Design

The backend returns findings and counts but does not compute a risk score. The frontend computes a 0–100 score from the real findings. Each finding is given points according to its severity, and the overall score is the mean of those points.

Severity	Points
Critical	90
High	70
Medium	45
Low	20
Unknown	10

Score range	Category
0–24	Low
25–49	Moderate
50–74	High
75–100	Critical

Table 2.2: Severity points and risk categories (frontend heuristic).

This is a heuristic, and we label it as one in the code. Section 4.4 discusses its weakness.

Chapter 3

Implementation

3.1 Backend

The backend is a FastAPI application in `backend/app/`. The entry point `main.py` creates the tables on startup, enables CORS so the frontend on port 5173 can call the API on port 8000, and defines the routes. `models.py` holds the SQLAlchemy models, `database.py` creates the engine and the per-request session, and the `services/` folder holds the parsing and vulnerability-checking logic.

3.1.1 API Endpoints

Endpoint	Purpose
`GET /health`	Liveness check
`POST /scan`	Upload a `package.json`, check it, store and return the result
`GET /scans`	List all scans (summary only), newest first
`GET /scans/{id}`	One scan with all of its findings
`DELETE /scans/{id}`	Delete a scan and its findings
`GET /stats`	Summary numbers of the latest scan
`GET /api/scan`	Parser-only check using the bundled sample file

Table 3.1: Backend API.

The upload endpoint rejects files that do not end in `.json` (HTTP 400), files over 1 MB (HTTP 413), and content that is not valid JSON (HTTP 400).

3.1.2 Parser

`parser.py` reads both `dependencies` and `devDependencies` and marks each entry as development or runtime. npm version specifiers such as `^4.17.15` or `~1.2` cannot be sent to OSV as they are, so `clean_version` converts them to a single concrete version.

Specifiers that cannot be resolved (`*`, `latest`, git and file URLs) return `None` and the dependency is reported as skipped.

Listing 3.1: Version cleaning in `parser.py`.

```
def clean_version(spec: str) -> str | None:
    if not isinstance(spec, str):
        return None
    if spec.startswith(("git", "http", "file:", "link:", "workspace:")):
        return None
    m = _VERSION_RE.search(spec)
    if not m:
        return None
    major, minor, patch, pre = m.groups()
    return f"{major}.{minor␣or␣0}.{patch␣or␣0}{pre␣or␣''}"
```

For a range such as `^4.16.0` this returns the lowest version the range allows, which is an approximation because no lockfile is available.

3.1.3 Vulnerability Checker

`vuln_checker.py` sends one POST request per dependency to the OSV query endpoint with the package name, the npm ecosystem, and the version. The requests run concurrently with `asyncio.gather`, and a semaphore limits the number in flight to 10 so that a large dependency list does not flood the API. Each request has a 15-second timeout. A network or HTTP error for one package is recorded as that package's error and does not abort the scan.

For each vulnerability returned, the checker extracts the identifier, summary, aliases (such as CVE numbers), the fixed versions for that package, and a severity. GitHub advisories carry a plain severity label, which is read and normalized: `MODERATE` is mapped to `MEDIUM`, and anything unrecognized becomes `UNKNOWN`.

Listing 3.2: Concurrent checking in `vuln_checker.py`.

```
async def check_all_dependencies(dependencies: list[dict]) -> list[dict]:
    sem = asyncio.Semaphore(MAX_CONCURRENT)
    async with httpx.AsyncClient(timeout=15.0) as client:
        return await asyncio.gather(
            *(_check_one(client, sem, d) for d in dependencies)
        )
```

3.1.4 Persistence

After checking, the `/scan` handler builds one `Scan` row with the counts per severity and one `Finding` row per vulnerability, then commits both in a single transaction. Fixed versions and aliases are stored as comma-separated strings and split back into lists when the scan is serialized.

3.2 Frontend

The frontend is a single-page React application with the following routes.

Route	Page
`/login, /register`	Authentication screens (mocked, see below)
`/projects`	List of the user's projects; create a new project
`/projects/:projectId/upload`	Drag-and-drop upload of `package.json` and Run scan
`/dashboard/:projectId`	Risk score, summary cards, priority fixes, dependency table, scan history

Table 3.2: Frontend routes.

Reusable components include the risk score card, stat cards, dependency table, priority fix list, risk history chart, file dropzone, severity badges, and a modal for creating a project. Pages other than login and register are wrapped in a `ProtectedRoute` that redirects unauthenticated users to the login page.

`scanApi.ts` contains the real backend calls through Axios. `mapScanToDashboard.ts` converts the backend's scan response into the shape the dashboard displays, groups findings by package and version, sorts packages by risk, selects the three most severe findings as priority fixes, and computes the risk score described in Section 2.7. Values the backend cannot provide (license, latest version, outdated status) are shown as unknown instead of being invented.

Mocked layer. Registration, login, and projects are stored in the browser's `localStorage` through `authService.ts` and `mockData.ts`. Passwords in this mock are not hashed and the tokens are not verified by any server; this layer exists only so the interface could be built and demonstrated before backend authentication exists.

3.3 Running the Project

`start.bat` (Windows) and `start.sh` (macOS/Linux) create the Python virtual environment, install backend and frontend dependencies, start Uvicorn on port 8000 and Vite on port 5173, and open the browser. The `stop` scripts shut both servers down. The repository includes `sample_package.json`, which contains deliberately old versions of `lodash` and `minimist` so that a demonstration scan produces findings.

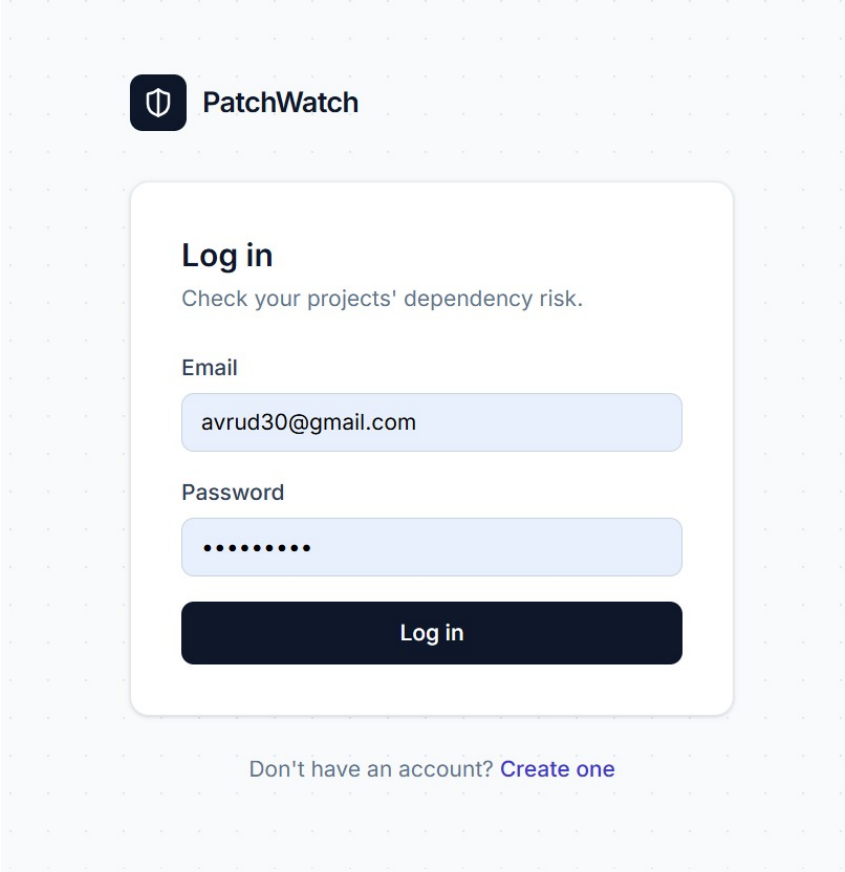


Figure 3.1: Login screen.

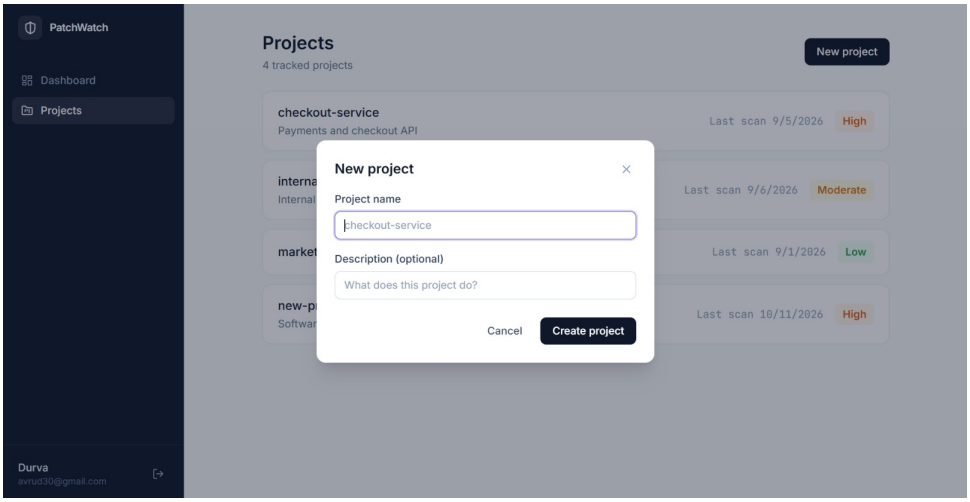


Figure 3.2: Projects page.

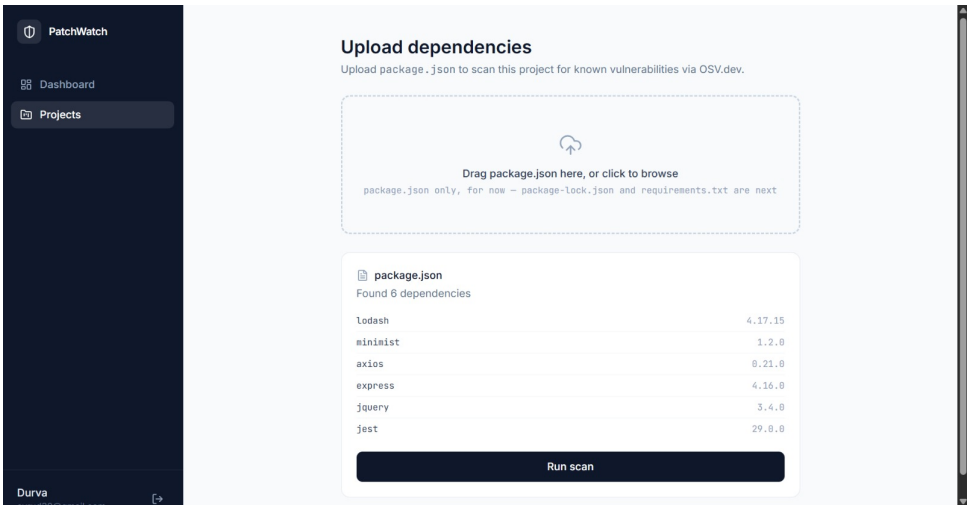


Figure 3.3: Upload page with a dropped `package.json`.

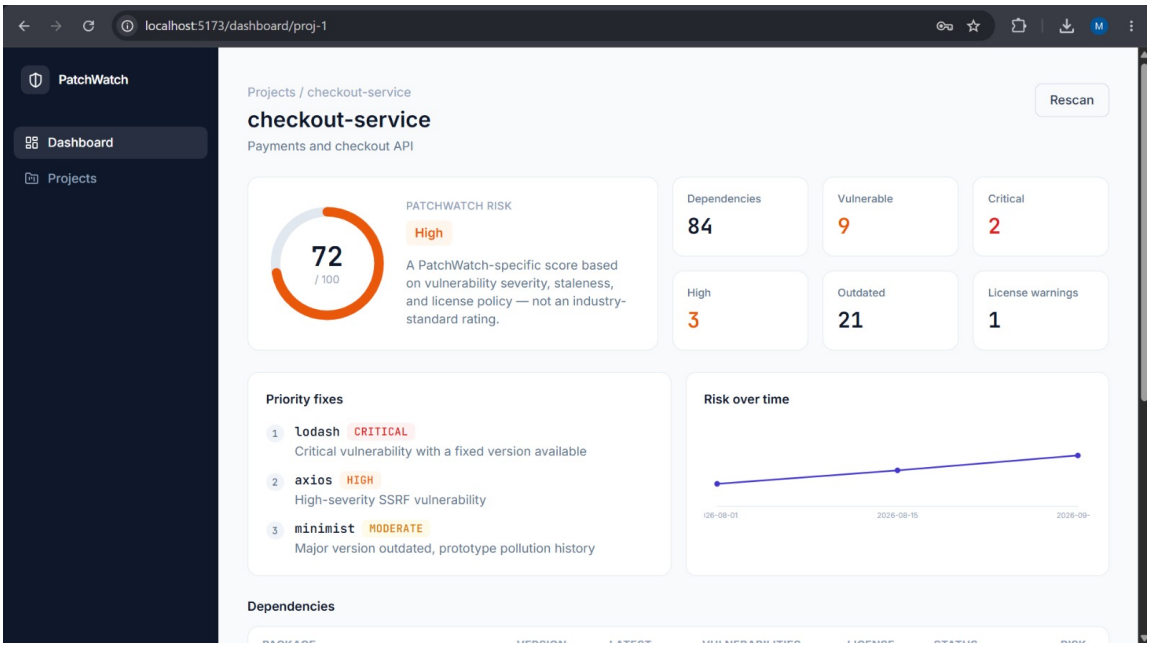


Figure 3.4: Dashboard after a real scan: risk score, summary cards, priority fixes, and dependency table.

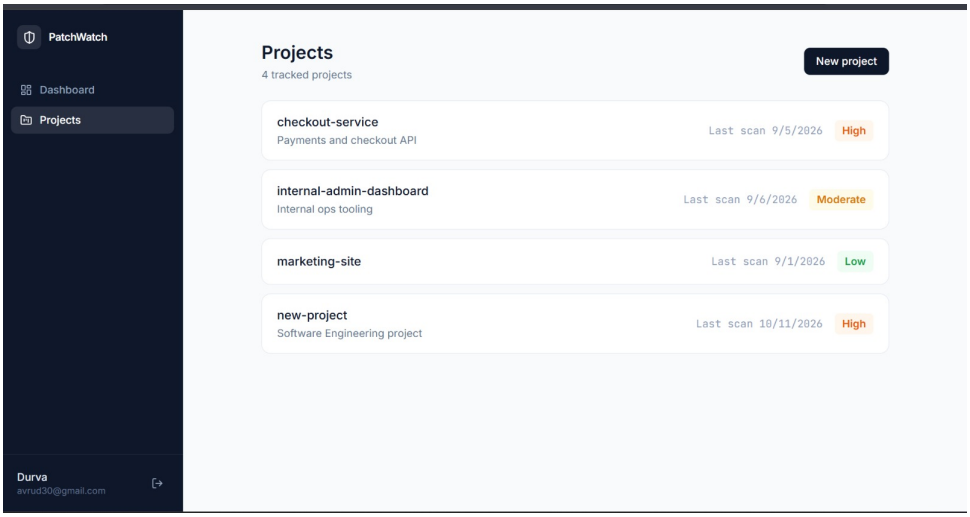


Figure 3.5: Scan history chart after a rescan.

Chapter 4

Results and Testing

4.1 Testing Strategy

Testing was kept practical and was split into two levels.

- **Automated backend tests (pytest).** Parser tests check version cleaning and the runtime/development split. API tests run against an in-memory SQLite database. They replace the OSV call with a fake so the tests are fast and do not need a network connection.
- **Manual end-to-end checks.** Because the automated tests mock OSV.dev, the real integration was checked by hand against the live API, using the sample file and the running frontend.

4.2 Automated Test Results

All four automated tests pass (`4 passed`).

#	Test	What it checks	Result
1	`test_clean_version`	`^4.17.15`, `~1.2`, `>=2` resolve to concrete versions; `*`, `latest`, and git URLs return `None`	Pass
2	`test_parse_includes_dev`	Both dependency sections are parsed and dev dependencies are flagged	Pass
3	`test_scan_saves_and_lists`	Upload saves a scan; list, detail, and stats endpoints return it; delete removes it and a second lookup returns 404	Pass
4	`test_bad_json_rejected`	Invalid JSON upload returns HTTP 400	Pass

Table 4.1: Automated test results.

4.3 Manual End-to-End Test Plan

#	Test case	Expected behaviour to verify during the demonstration
1	Scan `sample_package.json`	The scan completes and the dashboard displays findings and severity counts returned by the live OSV.dev service. Record actual counts and elapsed time during the run.
2	Upload a file with no known vulnerable dependencies	The scan completes and reports no vulnerability findings for the checked versions.
3	Upload a non-JSON file or invalid JSON	The API rejects the upload with an appropriate error; invalid JSON is expected to return HTTP 400.
4	Dependency with `*` or a git URL	The dependency is marked as skipped because a concrete version cannot be resolved by the parser.
5	Run without internet access	Network failures should be recorded as dependency-level errors where handled by the checker; verify that the application does not terminate unexpectedly.
6	Restart the backend and reload scan history	Previously saved scan records should remain available because the application persists scans in SQLite.
7	Rescan the same project	A new scan should be saved and, if the frontend history chart is connected to the returned scan history, a new history point should appear.

Table 4.2: Manual end-to-end test plan and expected behaviour. Actual outcomes must be recorded after execution.

4.4 Discussion

The central feature works: a real dependency file is parsed, checked against live data, stored, and displayed. We also want to state the limitations plainly.

1. **Version resolution is approximate.** Without a lockfile, a range like `^4.16.0` is checked as `4.16.0`, the lowest version it allows. The version npm would actually install may be newer and may not be affected. The scan can therefore over-report for ranges, and it cannot see the transitive dependencies at all.

2. **The risk score can mislead.** It is the average of per-finding points, so adding a low-severity finding to a project that already has a critical one lowers the score. A weighted or maximum-based score would be more defensible. Since it is computed in the browser, it is also not stored with the scan.

3. **Authentication and projects are not real.** They live in `localStorage`. The backend has no users, so every scan is global and any client can list or delete any scan. The CORS policy currently allows all origins. This is acceptable for a local demo and not for deployment.

4. **Automated tests do not cover OSV.dev.** The live integration, the severity extraction on real responses, and the frontend are only checked manually. There are no frontend tests.

5. **Single page of OSV results.** The checker reads only the first response page from OSV for each package.

6. **Scanning is synchronous.** The request waits for every lookup. This is fine for small files, but a very large dependency list would make the upload slow.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

PatchWatch turns a `package.json` into a prioritized view of known vulnerabilities. The scanning pipeline is real end to end: it parses the file, queries OSV.dev concurrently, normalizes severity, stores the results in SQLite, and drives a dashboard with history. The parts that depend on backend accounts are deliberately mocked and are labelled as such, and features we did not build are shown as unknown instead of being faked.

5.2 Future Work

- Real authentication (hashed passwords, JWT) and a `Project` table, so that scans belong to a user and a project.
- Parse `package-lock.json` to check exact installed versions and transitive dependencies; add `requirements.txt` support.
- Move the risk score to the backend, store it with each scan, and use a weighting that does not reward adding low-severity findings.
- Outdated-version detection and license analysis.
- Follow OSV response pagination and cache results to reduce repeated lookups.
- Run scans as background jobs for large projects.
- Frontend tests and a CI pipeline that runs the test suite on every push.
- Restrict CORS and move to PostgreSQL for deployment.

5.3 Lessons Learned

The project reinforced the value of developing and testing a system incrementally. Building the application in smaller stages made it possible to validate the parser, API, persistence layer, and dashboard integration separately instead of waiting until the end to

discover integration problems. We also learned that a frontend prototype can progress using a clearly isolated mock layer, but the report and demonstration must distinguish mock functionality from features backed by the server.

Integrating an external vulnerability service introduced concerns beyond simply making an API request. The application must handle timeouts and network failures, normalize severity labels, and explain when a dependency is skipped because its version cannot be resolved. Persistence was another important design consideration: saving scan records allows users to inspect previous results instead of losing them when a request finishes.

The project also highlighted the importance of testing both normal and failure cases. Automated tests can verify parsing and API behaviour in a repeatable way, while manual end-to-end checks are still needed to confirm the live OSV.dev integration and the complete user workflow. If we extended the project, we would prioritize exact installed-version data from lockfiles, server-backed authentication, and broader automated testing.

Bibliography

[1] OSV, *Open Source Vulnerabilities: API Documentation*. Available: https://osv.dev/docs/

[2] FastAPI, *FastAPI Documentation*. Available: https://fastapi.tiangolo.com/

[3] React, *React Documentation*. Available: https://react.dev/

[4] SQLAlchemy, *SQLAlchemy 2.0 Documentation*. Available: https://docs.sqlalchemy.org/

[5] pytest, *pytest Documentation*. Available: https://docs.pytest.org/